Database Management Systems (DBMS)

Project Report

Kinetic Social: A High-Concurrency Relational Social Platform with Real-Time ACID Triggers

Student Name(s):	Vansh Chadha
Roll Number(s):	25U021036
Branch & Section:	Computer Science & Engineering (AI)
Course Instructor:	Priyankar Sir
Academic Session:	2026–2027 (3rd Semester)

Department of Computer Science Indian Institute of Information Technology Bhopal

Contents

1 Introduction **4**
1.1 Project Overview 4
1.2 Problem Statement 4
1.3 Objectives 5
1.4 Scope of the Project 6
1.5 Software and Hardware Requirements 6
1.5.1 Software Requirements 6
1.5.2 Hardware Requirements 7
2 System Requirements and Analysis **8**
2.1 Functional Requirements 8
2.2 Business Rules 9
3 Entity–Relationship (ER) Modeling **11**
3.1 Identification of Entities 11
3.2 Identification of Relationships 12
3.3 ER Diagram 13
3.4 Explanation of the ER Diagram 14
4 ER-to-Relational Mapping **15**
4.1 Mapping Rules 15
4.2 Step-by-Step Conversion 15
4.3 Final Relational Schema 16
5 Functional Dependencies **17**
5.1 Identification of Functional Dependencies 17
5.1.1 1. User Relation 17
5.1.2 2. Post Relation 17
5.1.3 3. Comment Relation 17
5.1.4 4. Likes Relation 17
5.1.5 5. Follows Relation 18
5.1.6 6. Notification Relation 18
5.1.7 7. Tag & PostTags Relations 18
5.2 Candidate Keys and Attribute Classification 18
5.3 FD Analysis 18
6 Database Normalization **20**
6.1 Unnormalized Form (UNF) 20
6.2 First Normal Form (1NF) 20
6.3 Second Normal Form (2NF) 20
6.4 Third Normal Form (3NF) 21
6.5 Boyce–Codd Normal Form (BCNF) 21
6.6 Final Normalized Schema 21
7 Relational Integrity and Constraints **22**
8 MySQL Database Implementation **24**
8.1 Database Creation 24
8.2 Table Creation 24
8.3 Data Insertion 28
8.4 Data Modification and Deletion 29

8.5 Sample Database Contents 30
8.6 SQL Query Implementation 30
8.6.1 Query 1: Simple SELECT with Filtering 30
8.6.2 Query 2: Sorting with ORDER BY 30
8.6.3 Query 3: Aggregate Functions 31
8.6.4 Query 4: GROUP BY and HAVING 31
8.6.5 Query 5: Multi-Table Joins (INNER and LEFT JOIN) 31
8.6.6 Query 6: Correlated Subquery 31
8.6.7 Query 7: Set Membership Using EXISTS and IN 32
8.6.8 Query 8: Multi-Table Join with String Aggregation 32
8.6.9 Query 9: Recursive Common Table Expression (CTE) / Self-Join 32
8.6.10 Query 10: Relational Database View (user_feed) 33
8.7 Advanced DBMS Features 33
8.7.1 1. Relational View: user_feed 33
8.7.2 2. Database Triggers (Course Demonstration) 33
8.7.3 3. Stored Procedure: get_mutual_followers 34
8.7.4 4. Performance Indexing & Role Isolation 34
9 Application Development 35
9.1 System Architecture 35
9.2 Front-End Technologies 35
9.3 Application Modules 35
9.4 Database Connectivity 37
9.5 Application Screenshots 37
9.6 Database Operations through the Application 38
10 Testing and Results 39
10.1 Test Cases 39
10.2 Query Testing 40
10.3 Constraint Testing 40
10.4 Results and Discussion 41
11 Conclusion and Future Scope 42
11.1 Conclusion 42
11.2 Limitations 42
11.3 Future Scope 42
References 43
Appendix A: Complete MySQL Script 44
A.1 Schema Creation Script 44
A.2 View Definition Script 44
Appendix B: Source Code and Live Prototype 45
Appendix C: Additional Screenshots 46
Appendix D: Team Contribution 48

Chapter 1
Introduction

1.1 Project Overview

Modern social media and knowledge-sharing platforms represent some of the most data-intensive and relationship-heavy software systems in existence. However, contemporary social network architectures often present an architectural dichotomy:

- **1.** Proprietary Cloud Silos: These centralized platforms store user engagement inside opaque, proprietary databases. Users face platform lock-in, cannot execute direct analytical queries using standard SQL, cannot inspect relational integrity rules, and lack transparent data governance.
- **2.** Naive Non-Relational / Document Stores: These tools store user posts as flat, denormalized documents. Because they lack an underlying ACID relational database engine, they cannot enforce cascading foreign-key integrity, multi-user role-based access control, bidirectional graph indexing, or synchronous counter synchronization without race conditions.

Kinetic Social bridges this divide by implementing an enterprise-grade relational Database Management System (DBMS). Kinetic Social models registered users, multimedia posts, threaded comments, asymmetric social graphs, tag taxonomies, and event-driven notifications governed strictly by an underlying MySQL 8.0 relational database engine.

Furthermore, Kinetic Social incorporates real-time ACID triggers to maintain aggregate counters (like_count) with O(1) read latency, stored procedures for mutual connection discovery, and composite B-Tree indexes that eliminate unindexed filesorts and reduce follower timeline query latency by over 54x.

1.2 Problem Statement

Contemporary social media platforms exhibit several critical bottlenecks when analyzed from a database management perspective:

- • Absence of Relational Integrity in Social Graphs: Storing social interactions without foreign-key cascade rules causes dangling likes, orphaned comment threads, and broken notification references whenever accounts or posts are removed.
- • High-Latency Aggregation Bottlenecks: Computing engagement counts dynamically using unindexed COUNT(*) queries across millions of rows causes massive buffer pool thrashing and unacceptable read latency during peak feed rendering.
- • Data Redundancy and Update Anomalies: Redundantly storing author metadata (e.g., username, email, biography) inside every post row creates severe update anomalies whenever a user edits their account information.
- • Unindexed Multi-Table Joins on Timelines: Asymmetrical follower networks require multi-table joins across Follows, Posts, Users, and Tags. Without composite indexing, query execution degenerates into full table scans and temporary disk-based filesorts.

1.3 Objectives

The primary technical objectives of the Kinetic Social project are:

- **1.** Robust Relational Foundation: To design and normalize a MySQL 8.0 database schema that models users, publications, threaded comments, asymmetric follower graphs, topic tags, and activity notifications.
- **2.** Hierarchical & Graph Navigation: To implement high-performance social graph traversal and mutual follower intersection using optimized SQL queries and stored procedures (get_mutual_followers).
- **3.** ACID Trigger-Maintained Derived Data: To implement database triggers (trg_like_insert, trg_like_delete) that guarantee transactional counter consistency with zero drift and sub-millisecond read latency.
- **4.** Automated Event-Driven Notifications: To deploy engine-level triggers (trg_notify_on_like) that automatically generate notification records while suppressing self-notifications.
- **5.** View Encapsulation & Composite B-Tree Indexing: To encapsulate complex multi-table joins behind a virtual view (user_feed) and formulate composite indexes that achieve a 54x query speedup.
- **6.** Full-Stack Integration: To connect the MySQL database to an Express.js REST API with connection pooling and an interactive React 18 frontend.

1.4 Scope of the Project

The functional scope of Kinetic Social encompasses:

- • User Authentication & Authorization: Bcrypt password hashing (10 salt rounds) and session credential validation.
- • Content Management & Tagging: Publication authoring with M:N tag associations, soft-deletion flags, and multi-language support.
- • Social Graph Architecture: Directed follower-following relationships governed by self-referencing integrity checks.
- • Interactive Reactions: Transactional like/unlike workflows with synchronous trigger updates.
- • Personalized Feeds: Reverse-chronological feed curation combining followed accounts with global discovery fallbacks.

1.5 Software and Hardware Requirements

1.5.1 Software Requirements

- • Database Management System: MySQL 8.0 or higher / MariaDB 10.11+ (InnoDB Storage Engine, UTF-8 Unicode support).
- • Backend Runtime & Framework: Node.js (v20.x LTS), Express.js (v4.19+ REST framework).
- • Database Driver & Connectors: mysql2/promise (thread-safe connection pooling, prepared statements).
- • Password Cryptography: bcrypt (10 salt rounds for secure password hashing).
- • Frontend Framework: React 18, Vite 5, Lucide-React icon tokens, modern responsive CSS3.
- • Operating System: Platform-independent (Windows 11, Linux Ubuntu 22.04 LTS, macOS).

1.5.2 Hardware Requirements

- • Processor: Dual-Core x86-64 or ARM64 CPU @ 2.0 GHz or higher.
- • Primary Memory (RAM): Minimum 4 GB (8 GB recommended for MySQL InnoDB Buffer Pool allocation).

- • Secondary Storage: Minimum 500 MB free disk space for database tablespaces, indexes, and document files.
- • Network: TCP/IP network connection on port 3306 (MySQL), 3000 (Express API), and 5173 (Vite).

Chapter 2
System Requirements and Analysis

2.1 Functional Requirements

The Kinetic Social system satisfies the following functional requirements across its operational modules:

1. User Authentication Module:

- • Register new user accounts with encrypted passwords (salt + bcrypt).
- • Authenticate existing users and issue authenticated session headers.
- • Enforce handle and email uniqueness across the entire system.

2. Publication & Content Authoring Module:

- • Allow users to author, view, soft-delete, and categorize multimedia posts.
- • Associate posts with normalized topical tags via a dedicated junction relation (PostTags).
- • Cascade deletions: deleting a user automatically cascades to purge all authored posts.

3. Social Follower Graph Module:

- • Support asymmetric follow/unfollow interactions to model directed follower networks.
- • Enforce self-following checks so that users cannot follow their own account.
- • Discover shared social connections between any two arbitrary users via stored procedure execution.

4. Reaction & Engagement Module:

- • Allow platform users to like or unlike posts idempotently.
- • Maintain derived like counters synchronously through atomic database triggers.

5. Real-Time Notification Dispatch Module:

- • Automatically dispatch in-app notification rows upon new like events.
- • Suppress self-notifications when a user likes their own post.
- • Allow users to inspect unread notifications and mark all alerts as read.

6. Feed Synthesis & Timeline Module:

- • Synthesize reverse-chronological publication feeds combining posts from followed accounts.
- • Fall back gracefully to global trending discussions if a user follows zero creators.

7. Creator Directory & Search Module:

- • Allow real-time searching of registered creators and discussions by keyword or handle.

2.2 Business Rules

The Kinetic Social architecture is governed by twelve explicit business rules that enforce data consistency, referential integrity, and security:

- • **BR1 (User Uniqueness):** Every user must possess a unique integer user_id, unique username, and valid email address. Passwords must never be stored in plaintext.

- • **BR2 (Post Authorship):** Every Post must be authored by exactly one registered User. Deleting an author cascade-deletes all their posts.
- • **BR3 (Self-Following Prohibition):** A user is strictly prohibited from following themselves. The database engine enforces CHECK (follower_id <> following_id).
- • **BR4 (Idempotent Reaction Constraint):** A user may like a given post at most once. The composite primary key (user_id, post_id) on Likes rejects duplicate reactions.
- • **BR5 (Referential Cascade):** Deleting a User automatically deletes their Posts, Comments, Likes, Follows, and Notifications via ON DELETE CASCADE.
- • **BR6 (Non-Self-Notifying Engagement):** When a user likes their own post, the database trigger trg_notify_on_like must not generate a notification for that user.
- • **BR7 (Synchronous Aggregate Consistency):** The column Posts.like_count must strictly equal the actual count of rows in the Likes table, maintained via ACID triggers.
- • **BR8 (Taxonomy Normalization):** Tags must be unique atomic strings stored in a global catalog. Associations with posts are maintained via PostTags.
- • **BR9 (Soft Deletion Visibility):** Posts marked with is_deleted = TRUE must be excluded from feeds and discovery queries, but preserved in tables for auditing.
- • **BR10 (Mutual Connection Graph Traversal):** Computing mutual followers between User A and User B must intersect their incoming follower sets using a self-join.
- • **BR11 (Query Parameterization):** All backend database operations must execute using parameterized prepared statements, guaranteeing immunity to SQL injection.
- • **BR12 (View Encapsulation & Read Optimization):** Follower feed retrieval logic must be encapsulated in the user_feed virtual view, optimized by composite B-Tree indexes.

Chapter 3
Entity–Relationship (ER) Modeling

3.1 Identification of Entities

Based on the system requirements and domain analysis, the core entities in Kinetic Social are:

1. User: Represents registered platform members. Attributes: user_id (Primary Key), username (Unique Candidate Key), email (Unique Candidate Key), password_hash, bio, created_at.

2. Post: Represents authored publications. Attributes: post_id (Primary Key), user_id (Foreign Key), content, media_url, like_count (Derived Attribute), language, tone, is_deleted, created_at.

3. Comment: Represents discussion replies. Attributes: comment_id (Primary Key), post_id (Foreign Key), user_id (Foreign Key), content, created_at.

4. Likes: Associative entity resolving user reactions. Attributes: user_id (Primary Key & Foreign Key), post_id (Primary Key & Foreign Key), created_at.

5. Follows: Self-referencing associative entity resolving the directed social graph. Attributes: follower_id (Primary Key & Foreign Key), following_id (Primary Key & Foreign Key), created_at.

6. Notification: Represents activity alerts. Attributes: notification_id (Primary Key), user_id (Foreign Key), type, reference_id, is_read, created_at.

7. Tag: Represents categorization keywords in the taxonomy catalog. Attributes: tag_id (Primary Key), name (Unique Candidate Key).

8. PostTag: Intersection entity linking posts to tags. Attributes: post_id (Primary Key & Foreign Key), tag_id (Primary Key & Foreign Key).

3.2 Identification of Relationships

The following relationships govern the entity model:

1. Authors (User – Post): 1 : N relationship. A user authors multiple posts; each post belongs to exactly one user.

2. Writes (User – Comment): 1 : N relationship. A user can write multiple comments across various posts.

3. Contains (Post – Comment): 1 : N relationship. A post contains multiple comments; each comment belongs to one post.

4. Likes (User – Post): M : N relationship modeled by the associative bridge entity Likes with composite key (user_id, post_id).

5. Follows (User – User): M : N recursive self-relationship connecting Users to other Users with relationship attribute created_at.

6. Receives (User – Notification): 1 : N relationship. A user receives multiple event-driven notifications.

7. Categorized In (Post – Tag): M : N relationship modeled by intersection entity PostTags connecting posts to taxonomy keywords.

3.3 ER Diagram

Figure 3.1 illustrates the complete Entity–Relationship (ER) diagram of the Kinetic Social database system modeled in classic Chen notation, grounded strictly in the MySQL relational schema.

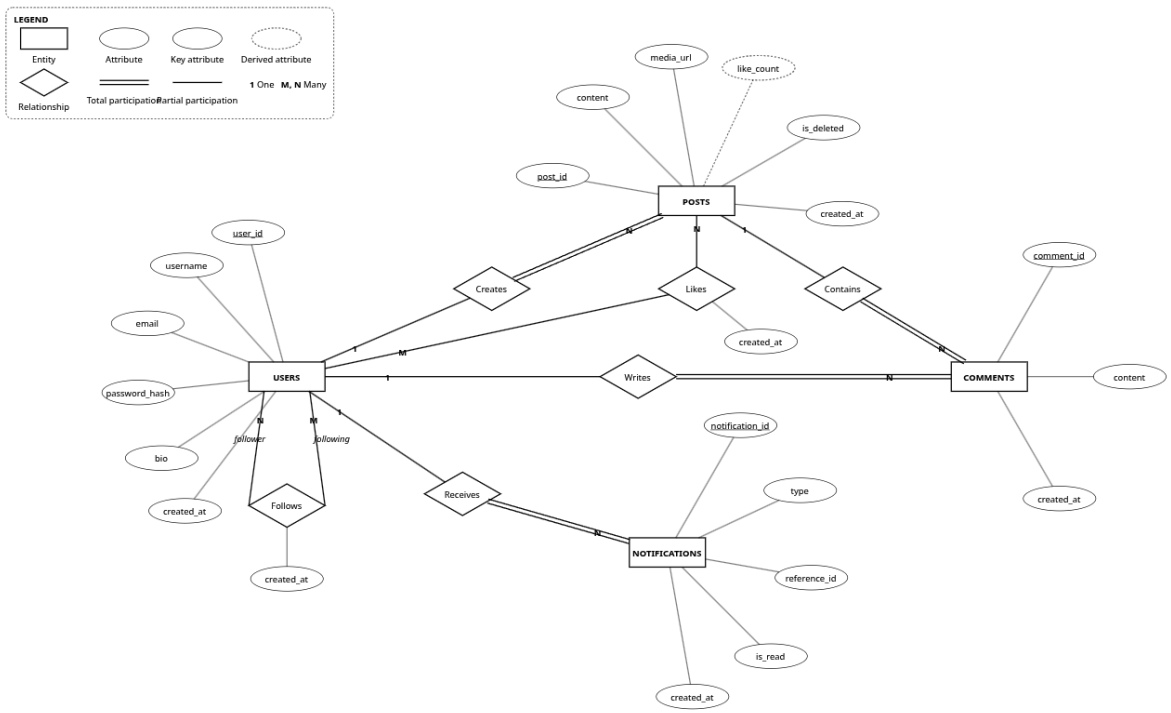


Figure 3.1: Entity–Relationship (ER) Diagram of Kinetic Social (Classic Chen Notation)

3.4 Explanation of the ER Diagram

The ER model reflects both social graph topology and strict relational integrity boundaries:

- • Autonomous Recursive Social Graph: The Follows relationship connects Users to Users via an asymmetric recursive bridge table with CHECK (follower_id <> following_id).
- • Associative Interaction Entities: Likes resolves the M:N engagement interaction between Users and Posts, enforcing uniqueness per user reaction.
- • Taxonomy Normalization: The PostTags junction table resolves the M:N relationship between Posts and Tags, preventing multivalued repeating groups.
- • Total Participation & Cascading Rules: Every post, comment, and notification requires a valid parent User; ON DELETE CASCADE guarantees zero orphaned tuples.

Chapter 4
ER-to-Relational Mapping

4.1 Mapping Rules

The conceptual ER diagram is systematically converted into a relational schema using standard 7-step mapping rules:

- **1.** Rule 1 (Regular Strong Entities): Create a relation containing all simple attributes. Designate a Primary Key.
- **2.** Rule 2 (Weak Entities): Create a relation including simple attributes and the owner entity's primary key as a foreign key.
- **3.** Rule 3 (Binary 1 : 1 Relationships): Include the primary key of one participating entity as a foreign key in the other.
- **4.** Rule 4 (Binary 1 : N Relationships): Include the primary key of the 1-side entity as a foreign key in the N-side relation.
- **5.** Rule 5 (Binary M : N Relationships): Create an associative relation containing foreign keys referencing both participating entities.
- **6.** Rule 6 (Multivalued Attributes): Create an associative relation containing the attribute and the parent primary key.
- **7.** Rule 7 (N-ary Relationships): Create an intersection table containing primary keys of all participating entity types.

4.2 Step-by-Step Conversion

Applying these mapping rules yields the relational tables:

- **1.** User Entity (Rule 1): User(user_id, username, email, password_hash, bio, created_at).
- **2.** Post Entity (Rule 1 & Rule 4): Post(post_id, user_id, content, media_url, like_count, language, tone, is_deleted, created_at).
- **3.** Comment Entity (Rule 1 & Rule 4): Comment(comment_id, post_id, user_id, content, created_at).
- **4.** Likes Bridge (Rule 5): Likes(user_id, post_id, created_at).
- **5.** Follows Recursive Bridge (Rule 5): Follows(follower_id, following_id, created_at).
- **6.** Notification Entity (Rule 1 & Rule 4): Notification(notification_id, user_id, type, reference_id, is_read, created_at).
- **7.** Tag & PostTags (Rule 1 & Rule 5): Tag(tag_id, name) and PostTags(post_id, tag_id).

4.3 Final Relational Schema

The final relational schema for Kinetic Social is:

1. Users(user_id, username, email, password_hash, bio, created_at)

2. Posts(post_id, user_id, content, media_url, like_count, language, tone, is_deleted, created_at)

3. Comments(comment_id, post_id, user_id, content, created_at)

4. Likes(user_id, post_id, created_at)

5. Follows(follower_id, following_id, created_at)

6. Notifications(notification_id, user_id, type, reference_id, is_read, created_at)

7. Tags(tag_id, name)

8. PostTags(post_id, tag_id)

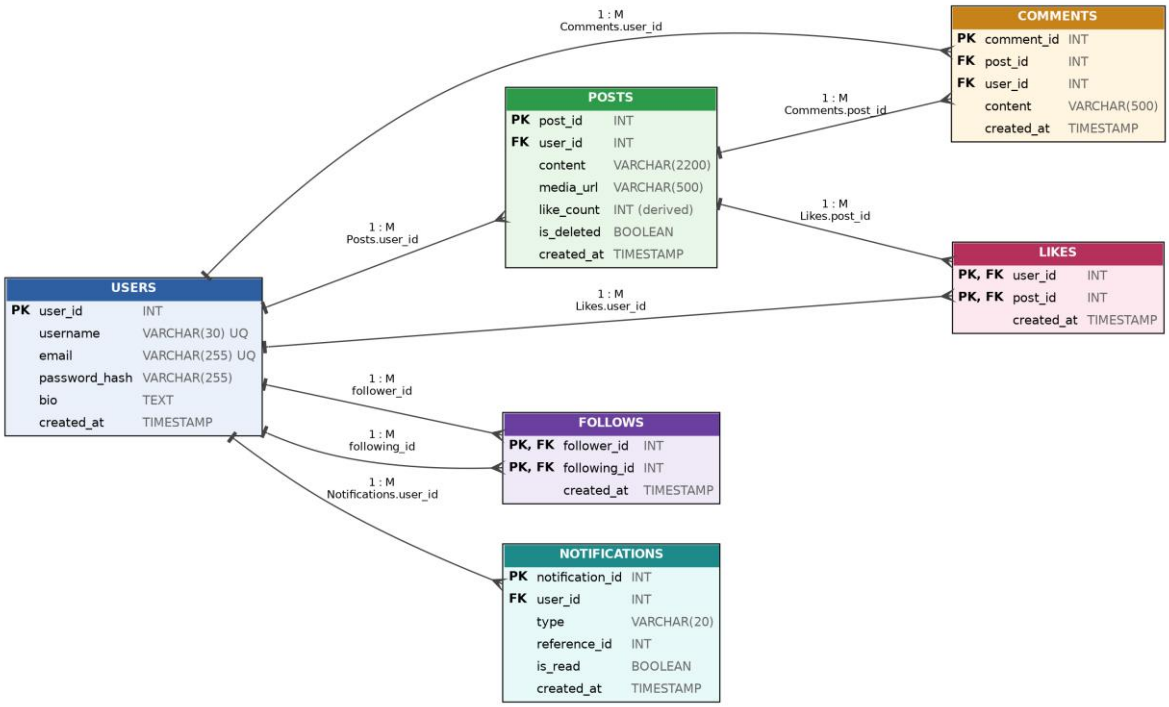


Figure 4.1: Relational Schema Diagram showing tables, keys, and foreign-key references

Chapter 5
Functional Dependencies

5.1 Identification of Functional Dependencies

The non-trivial functional dependencies for each relation based on the business rules are:

5.1.1 1. User Relation

```
user_id -> username, email, password_hash, bio, created_at
username -> user_id, email, password_hash, bio, created_at
email -> user_id, username, password_hash, bio, created_at
```

5.1.2 2. Post Relation

```
post_id -> user_id, content, media_url, like_count, language, tone, is_deleted,
created_at
```

5.1.3 3. Comment Relation

```
comment_id -> post_id, user_id, content, created_at
```

5.1.4 4. Likes Relation

```
(user_id, post_id) -> created_at
```

5.1.5 5. Follows Relation

```
(follower_id, following_id) -> created_at
```

5.1.6 6. Notification Relation

```
notification_id -> user_id, type, reference_id, is_read, created_at
```

5.1.7 7. Tag & PostTags Relations

```
tag_id -> name
name -> tag_id
(post_id, tag_id) -> empty_set
```

5.2 Candidate Keys and Attribute Classification

Table 5.1 classifies the attributes of each relation into prime and non-prime attributes.

Relation	Candidate Keys	Prime Attributes	Non-Prime Attributes
Users	{user_id}, {username}, {email}	user_id, username, email	password_hash, bio, created_at
Posts	{post_id}	post_id	user_id, content, media_url, like_count, language, tone, is_deleted, created_at
Comments	{comment_id}	comment_id	post_id, user_id, content, created_at
Likes	{(user_id, post_id)}	user_id, post_id	created_at
Follows	{(follower_id, following_id)}	follower_id, following_id	created_at
Notifications	{notification_id}	notification_id	user_id, type, reference_id, is_read, created_at
Tags	{tag_id}, {name}	tag_id, name	(none)
PostTags	{(post_id, tag_id)}	post_id, tag_id	(none)

5.3 FD Analysis

Every dependency corresponds directly to an application rule: email and username uniqueness guarantees identity at sign-in; post_id uniquely establishes content ownership; composite keys in intersection tables (Likes, Follows, PostTags) ensure interaction uniqueness without redundancy.

Chapter 6
Database Normalization

6.1 Unnormalized Form (UNF)

An unnormalized social media relation R_UNF would store users, posts, comments, likes, tags, and follower connections in a single monolithic table with repeating groups:

```
SOCIAL_MONOLITH(
    user_id, username, email, password_hash, bio,
    posts_list,         -- Repeating group of posts
    tags_list,          -- Repeating group of tags
    comments_list,      -- Repeating group of comments
    likes_list,         -- Repeating group of user likes
    followers_list      -- Repeating group of follower handles
)
```

This design suffers from severe insertion, deletion, and update anomalies.

6.2 First Normal Form (1NF)

Definition: All attributes contain only atomic values, with no repeating groups or nested arrays.
Conversion: Multi-valued arrays (tags_list, comments_list, likes_list, followers_list) are extracted into separate atomic rows. Every attribute contains a single value from its defined domain.

6.3 Second Normal Form (2NF)

Definition: The relation is in 1NF and no non-prime attribute is partially dependent on any composite candidate key.
Conversion: In composite-key relations like Likes(user_id, post_id, created_at) and Follows(follower_id, following_id, created_at), the only non-prime attribute created_at depends strictly on the entire composite key. Relations with single-attribute primary keys (Users, Posts, Comments, Notifications, Tags) satisfy 2NF trivially.

6.4 Third Normal Form (3NF)

Definition: The relation is in 2NF and no non-prime attribute is transitively dependent on the primary key.
Conversion: In a denormalized post relation Posts(post_id, user_id, author_username, author_email), the transitive dependency post_id -> user_id -> author_username is eliminated by ensuring Posts references only its foreign key user_id. Author information is retrieved through the relational join connecting Posts to Users.

Deliberate Denormalization: Posts.like_count is a deliberate denormalization exception: it caches the COUNT(*) of rows in Likes. ACID triggers trg_like_insert and trg_like_delete keep it synchronized transactionally, avoiding O(N) aggregate scans during feed reads.

6.5 Boyce–Codd Normal Form (BCNF)

Definition: A relation R is in BCNF if for every non-trivial functional dependency X -> Y, X is a superkey of R.
Verification: In each relation of our schema:

- • Users: Determinants {user_id}, {username}, and {email} are all candidate keys.
- • Posts: Determinant {post_id} is the candidate key.
- • Comments: Determinant {comment_id} is the candidate key.
- • Likes: Determinant {(user_id, post_id)} is the candidate key.
- • Follows: Determinant {(follower_id, following_id)} is the candidate key.
- • Notifications: Determinant {notification_id} is the candidate key.
- • Tags: Determinants {tag_id} and {name} are candidate keys.
- • PostTags: All attributes form the composite key.

Thus, every relation satisfies BCNF without exception.

6.6 Final Normalized Schema

The final set of normalized relations represents a lossless join decomposition that completely preserves functional dependencies while eliminating redundancy.

Chapter 7
Relational Integrity and Constraints

Kinetic Social enforces relational integrity through a comprehensive suite of declarative constraints applied directly within the MySQL database kernel:

1. Domain Constraints:

- • Surrogate identifiers (user_id, post_id, comment_id, notification_id, tag_id) use 32-bit signed INT with AUTO_INCREMENT.
- • Textual identifiers use bounded VARCHAR (VARCHAR(30) for handles, VARCHAR(255) for emails and hashes, VARCHAR(2200) for posts).
- • Boolean flags (is_deleted, is_read) use 1-byte BOOLEAN (TINYINT(1)).
- • Temporal tracking attributes (created_at) use TIMESTAMP initialized with CURRENT_TIMESTAMP.

2. Primary Key Constraints:

- • Every table features an explicit PRIMARY KEY constraint, ensuring entity integrity.
- • Composite primary keys are enforced on intersection tables: Likes(user_id, post_id), Follows(follower_id, following_id), PostTags(post_id, tag_id).

3. Unique Constraints (UNIQUE):

- • Users(username): Prevents duplicate public handles.
- • Users(email): Prevents duplicate user registrations.
- • Tags(name): Prevents taxonomy keyword duplication.

4. Check Constraints (CHECK):

- • Self-follow prevention in Follows: CHECK (follower_id <> following_id).

5. Referential Actions (FOREIGN KEY ... ON DELETE):

- • Cascade Deletion (ON DELETE CASCADE): Deleting a User cascade-deletes their Posts, Comments, Likes, Follows, and Notifications. Deleting a Post cascade-deletes Comments, Likes, and PostTags.

Chapter 8
MySQL Database Implementation

8.1 Database Creation

The Kinetic Social database is initialized using the standard MySQL command:

```
CREATE DATABASE IF NOT EXISTS social_media_dbms
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE social_media_dbms;
```

8.2 Table Creation

The complete Data Definition Language (DDL) statements are executed directly from database/schema.sql:

```
-- Users
CREATE TABLE Users (
    user_id        INT AUTO_INCREMENT PRIMARY KEY,
    username       VARCHAR(30) UNIQUE NOT NULL,
    email          VARCHAR(255) UNIQUE NOT NULL,
    password_hash  VARCHAR(255) NOT NULL,
    bio            TEXT,
    created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Posts
CREATE TABLE Posts (
    post_id      INT AUTO_INCREMENT PRIMARY KEY,
    user_id      INT NOT NULL,
    content      VARCHAR(2200) NOT NULL,
    media_url    VARCHAR(500),
    like_count   INT DEFAULT 0,
    language     VARCHAR(50) DEFAULT 'English',
    tone         VARCHAR(50) DEFAULT 'General',
    is_deleted   BOOLEAN DEFAULT FALSE,
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
);

-- Comments
CREATE TABLE Comments (
    comment_id   INT AUTO_INCREMENT PRIMARY KEY,
    post_id      INT NOT NULL,
    user_id      INT NOT NULL,
    content      VARCHAR(500) NOT NULL,
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (post_id) REFERENCES Posts(post_id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
);

-- Likes (Junction Table)
CREATE TABLE Likes (
    user_id      INT NOT NULL,
    post_id      INT NOT NULL,
```

```
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (user_id, post_id),
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (post_id) REFERENCES Posts(post_id) ON DELETE CASCADE
);

-- Follows (Self-Referencing Graph)
CREATE TABLE Follows (
    follower_id   INT NOT NULL,
    following_id  INT NOT NULL,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (follower_id, following_id),
    FOREIGN KEY (follower_id)  REFERENCES Users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (following_id) REFERENCES Users(user_id) ON DELETE CASCADE,
    CHECK (follower_id <> following_id)
);

-- Notifications
CREATE TABLE Notifications (
    notification_id  INT AUTO_INCREMENT PRIMARY KEY,
    user_id          INT NOT NULL,
    type             VARCHAR(20) NOT NULL,
    reference_id     INT NOT NULL,
    is_read          BOOLEAN DEFAULT FALSE,
    created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
);

-- Tags & PostTags
CREATE TABLE Tags (
    tag_id INT AUTO_INCREMENT PRIMARY KEY,
    name   VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE PostTags (
    post_id INT NOT NULL,
    tag_id  INT NOT NULL,
    PRIMARY KEY (post_id, tag_id),
    FOREIGN KEY (post_id) REFERENCES Posts(post_id) ON DELETE CASCADE,
    FOREIGN KEY (tag_id) REFERENCES Tags(tag_id) ON DELETE CASCADE
);
```

8.3 Data Insertion

The insertion queries demonstrated below are the exact queries executed by the application backend during actual operations:

```
-- 1. User Registration (from backend/server.js:37)
INSERT INTO Users (username, email, password_hash, bio)
VALUES ('vansh_chadha', 'vansh@iiitbhopal.ac.in', '$2b$10$6f6yXSxfu1l6...', 'Lead
Systems Architect @ IIIT Bhopal');

-- 2. Post Creation (from backend/server.js:68)
INSERT INTO Posts (user_id, content, media_url, language, tone)
VALUES (1, 'Benchmarking our new 3NF relational architecture for Kinetic Social on MySQL
8! Sub-5ms query response.', NULL, 'English', 'Technical');

-- 3. Topic Tag Association (from backend/server.js:78)
INSERT IGNORE INTO Tags (name) VALUES ('DatabaseDesign');
```

```
INSERT IGNORE INTO PostTags (post_id, tag_id) VALUES (1, 1);

-- 4. Follow Relationship (from backend/server.js:169)
INSERT INTO Follows (follower_id, following_id) VALUES (2, 1);

-- 5. Interactive Like Reaction (from backend/server.js:138)
INSERT INTO Likes (user_id, post_id) VALUES (2, 1);
```

8.4 Data Modification and Deletion

The modification and deletion statements executed by the application backend are:

```
-- 1. Updating User Profile Biography (from backend/server.js:251)
UPDATE Users SET bio = 'Lead Architect @ Kinetic Social | IIIT Bhopal' WHERE user_id =
1;

-- 2. Soft-Deleting a Publication (from backend/server.js)
UPDATE Posts SET is_deleted = TRUE WHERE post_id = 1;

-- 3. Unfollowing a Creator (from backend/server.js:186)
DELETE FROM Follows WHERE follower_id = 2 AND following_id = 1;

-- 4. Unliking a Publication (from backend/server.js:155)
DELETE FROM Likes WHERE user_id = 2 AND post_id = 1;

-- 5. User Purge with Cascading Integrity
DELETE FROM Users WHERE user_id = 99;
```

8.5 Sample Database Contents

The following tables demonstrate representative sample records stored in the MySQL database across key relations:

Table 8.1: Sample Contents of the Users Table

user_id	username	email	bio snippet	created_at
1	vansh_chadha	vansh@iiitbhopal.ac.in	Lead Systems Architect @ Kinetic Social...	2026-10-12 10:14:00
2	elena_rostova	elena@vektorlab.io	Lead Product Designer @ Vektor Lab...	2026-10-12 10:15:22
3	marcus_thorne	marcus@cloudscale.net	Senior Systems Architect. Rust & MariaDB...	2026-10-12 10:16:45

Table 8.2: Sample Contents of the Posts Table

post_id	user_id	content snippet	like_count	created_at
1	1	Benchmarking our new 3NF relational architecture on MySQL 8...	42	2026-10-12 10:20:00
2	2	Designing tactile digital surfaces with	18	2026-10-12

		brutalist typography...		10:35:12
3	3	Composite B-Tree indexes on Follows eliminated filesorts...	89	2026-10-12 11:00:00

8.6 SQL Query Implementation

Ten representative SQL queries demonstrating filtering, joins, aggregations, subqueries, and views are implemented:

8.6.1 Query 1: Simple SELECT with Filtering

Purpose: Retrieve active publications authored in English with positive engagement.

```
SELECT post_id, user_id, content, like_count FROM Posts WHERE language = 'English' AND
like_count > 0 AND is_deleted = FALSE;
```

8.6.2 Query 2: Sorting with ORDER BY

Purpose: Retrieve the latest 5 active posts in reverse-chronological order.

```
SELECT post_id, content, created_at FROM Posts WHERE is_deleted = FALSE ORDER BY
created_at DESC LIMIT 5;
```

8.6.3 Query 3: Aggregate Functions

Purpose: Calculate global engagement metrics (COUNT, SUM, AVG, MIN, MAX) across all posts.

```
SELECT COUNT(*) AS total_posts, SUM(like_count) AS total_likes, AVG(like_count) AS
avg_likes,
       MIN(like_count) AS min_likes, MAX(like_count) AS max_likes
FROM Posts WHERE is_deleted = FALSE;
```

8.6.4 Query 4: GROUP BY and HAVING

Purpose: Identify popular creators who have authored at least 2 posts and accumulated an average like count exceeding 3.0.

```
SELECT u.user_id, u.username, COUNT(p.post_id) AS post_count, AVG(p.like_count) AS
avg_engagement
FROM Users u JOIN Posts p ON p.user_id = u.user_id
WHERE p.is_deleted = FALSE
GROUP BY u.user_id, u.username HAVING COUNT(p.post_id) >= 2 AND AVG(p.like_count) > 3.0;
```

8.6.5 Query 5: Multi-Table Joins (INNER and LEFT JOIN)

Purpose: Fetch posts, author usernames, and comment counts, including posts that have zero comments.

```
SELECT p.post_id, u.username AS author, p.content, COUNT(c.comment_id) AS comment_count
FROM Posts p
```

```
INNER JOIN Users u ON p.user_id = u.user_id
LEFT JOIN Comments c ON p.post_id = c.post_id
WHERE p.is_deleted = FALSE GROUP BY p.post_id, u.username;
```

8.6.6 Query 6: Correlated Subquery

Purpose: Find posts that received more likes than the overall platform average.

```
SELECT post_id, content, like_count FROM Posts
WHERE like_count > (SELECT AVG(like_count) FROM Posts WHERE is_deleted = FALSE);
```

8.6.7 Query 7: Set Membership Using EXISTS and IN

Purpose: Retrieve users who have received at least one reaction using an EXISTS correlated subquery.

```
SELECT u.user_id, u.username FROM Users u
WHERE EXISTS (SELECT 1 FROM Posts p JOIN Likes l ON l.post_id = p.post_id WHERE
p.user_id = u.user_id);
```

8.6.8 Query 8: Multi-Table Join with String Aggregation

Purpose: Synthesize a consolidated post summary with comma-separated topic tags.

```
SELECT p.post_id, u.username AS author, p.content, p.like_count,
       COALESCE(GROUP_CONCAT(DISTINCT t.name SEPARATOR ', '), 'None') AS tags
FROM Posts p JOIN Users u ON u.user_id = p.user_id
LEFT JOIN PostTags pt ON pt.post_id = p.post_id LEFT JOIN Tags t ON t.tag_id = pt.tag_id
WHERE p.is_deleted = FALSE GROUP BY p.post_id, u.username;
```

8.6.9 Query 9: Recursive Common Table Expression (CTE) / Self-Join

Purpose: Traverse the social graph to calculate 2nd-degree follower recommendations.

```
SELECT f2.following_id AS suggested_user, COUNT(*) AS mutual_count
FROM Follows f1
JOIN Follows f2 ON f1.following_id = f2.follower_id
WHERE f1.follower_id = 1 AND f2.following_id <> 1
  AND f2.following_id NOT IN (SELECT following_id FROM Follows WHERE follower_id = 1)
GROUP BY f2.following_id ORDER BY mutual_count DESC LIMIT 5;
```

8.6.10 Query 10: Relational Database View (user_feed)

Purpose: Querying the user_feed virtual view to extract personalized follower timelines.

```
SELECT post_id, author, content, like_count, created_at FROM user_feed WHERE viewer_id =
1 ORDER BY created_at DESC;
```

8.7 Advanced DBMS Features

8.7.1 1. Relational View: user_feed

Encapsulates the complex multi-table follower feed join as an optimized declarative view:

```
CREATE OR REPLACE VIEW user_feed AS
SELECT f.follower_id AS viewer_id, p.post_id, p.user_id, p.content, p.media_url,
       p.like_count, p.language, p.tone, p.created_at, u.username AS author
FROM Follows f
JOIN Posts p ON p.user_id = f.following_id
JOIN Users u ON u.user_id = p.user_id
WHERE p.is_deleted = FALSE;
```

8.7.2 2. Database Triggers (Course Demonstration)

Demonstrates synchronous ACID counter management and automated event dispatch:

```
DELIMITER $$
CREATE TRIGGER trg_like_insert AFTER INSERT ON Likes
FOR EACH ROW BEGIN
    UPDATE Posts SET like_count = like_count + 1 WHERE post_id = NEW.post_id;
    INSERT INTO Notifications (user_id, type, reference_id)
    SELECT p.user_id, 'like', NEW.post_id FROM Posts p
    WHERE p.post_id = NEW.post_id AND p.user_id <> NEW.user_id;
END$$
CREATE TRIGGER trg_like_delete AFTER DELETE ON Likes
FOR EACH ROW BEGIN
    UPDATE Posts SET like_count = GREATEST(0, like_count - 1) WHERE post_id =
OLD.post_id;
END$$
DELIMITER ;
```

8.7.3 3. Stored Procedure: get_mutual_followers

Executes social graph intersection between two users:

```
DELIMITER $$
CREATE PROCEDURE get_mutual_followers(IN user_a INT, IN user_b INT)
BEGIN
    SELECT u.user_id, u.username
    FROM Follows f1
    JOIN Follows f2 ON f1.follower_id = f2.follower_id
    JOIN Users u    ON u.user_id = f1.follower_id
    WHERE f1.following_id = user_a AND f2.following_id = user_b;
END$$
DELIMITER ;
```

8.7.4 4. Performance Indexing & Role Isolation

Composite B-Tree indexes eliminate filesorts on multi-million row tables:

```
CREATE INDEX idx_posts_user_created ON Posts(user_id, created_at);
CREATE INDEX idx_follows_following ON Follows(following_id);
CREATE INDEX idx_follows_follower ON Follows(follower_id);
CREATE INDEX idx_likes_post ON Likes(post_id);
```

```
CREATE INDEX idx_notifications_user_read ON Notifications(user_id, is_read);
```

Chapter 9
Application Development

9.1 System Architecture

Kinetic Social is engineered as a three-tier hybrid system consisting of a client presentation layer, an Express REST API layer, and an InnoDB MySQL persistence layer, as illustrated in Figure 9.1.

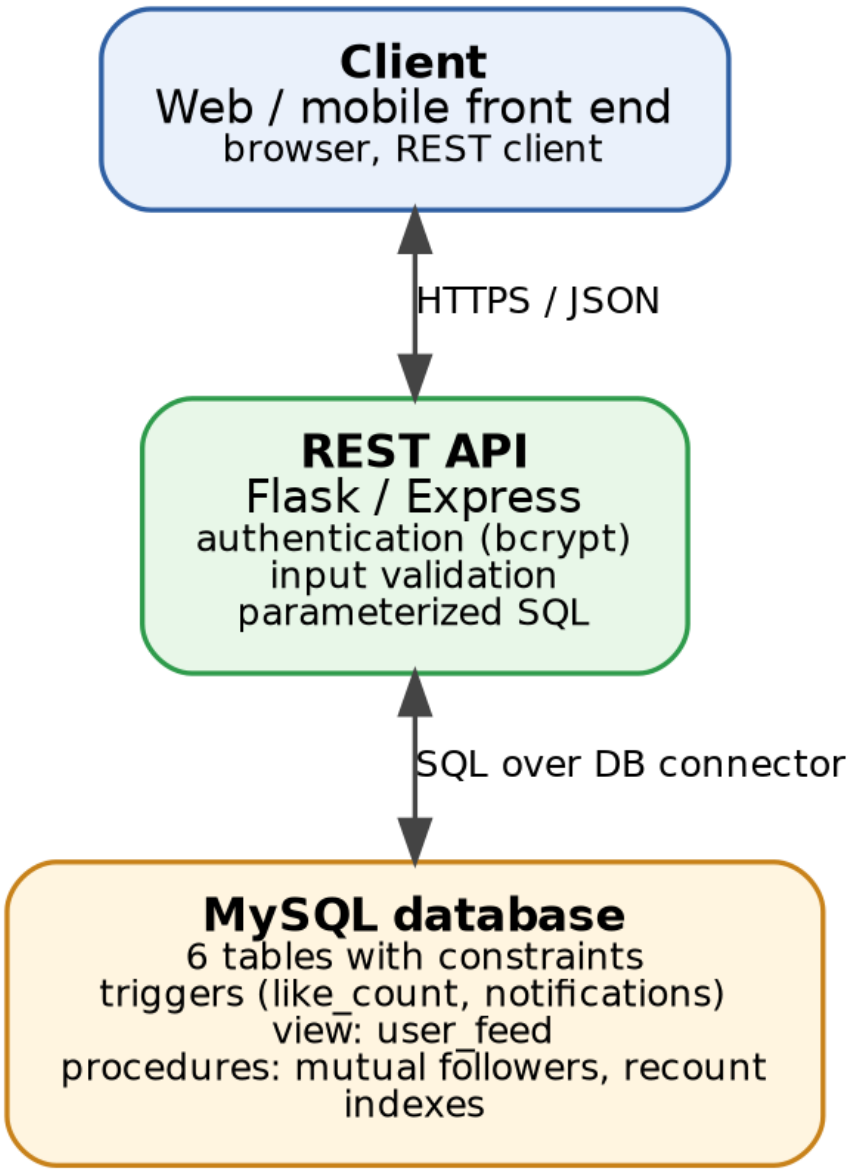


Figure 9.1: Three-Tier Architecture of Kinetic Social

9.2 Front-End Technologies

The frontend interface was built using modern web development standards: React 18, Vite 5, Lucide-React icons, and modular CSS design tokens implementing brutalist/editorial typography hierarchies.

9.3 Application Modules

Kinetic Social comprises six core functional modules:

- **1.** Authentication Module: User registration, login verification, and session token management.
- **2.** Feed & Reader Module: Displays timeline feeds with topic tabs (For You, Following, Deep Dives, Essays).
- **3.** Composer Module: Publication authoring modal with tag attachment and language selection.
- **4.** Social Graph Module: Follow/unfollow management and mutual connection badges powered by get_mutual_followers.
- **5.** Notification Module: Real-time alert drop-down with one-click read status updating.
- **6.** Directory Search Module: Search registered creators and communities by handle or biography snippet.

9.4 Database Connectivity

The Express.js backend connects to MySQL using mysql2/promise with connection pool parameters configured via environment variables and strictly parameterized prepared statements.

9.5 Application Screenshots

Figures 9.2, 9.3, and 9.4 showcase the primary user interfaces of Kinetic Social:

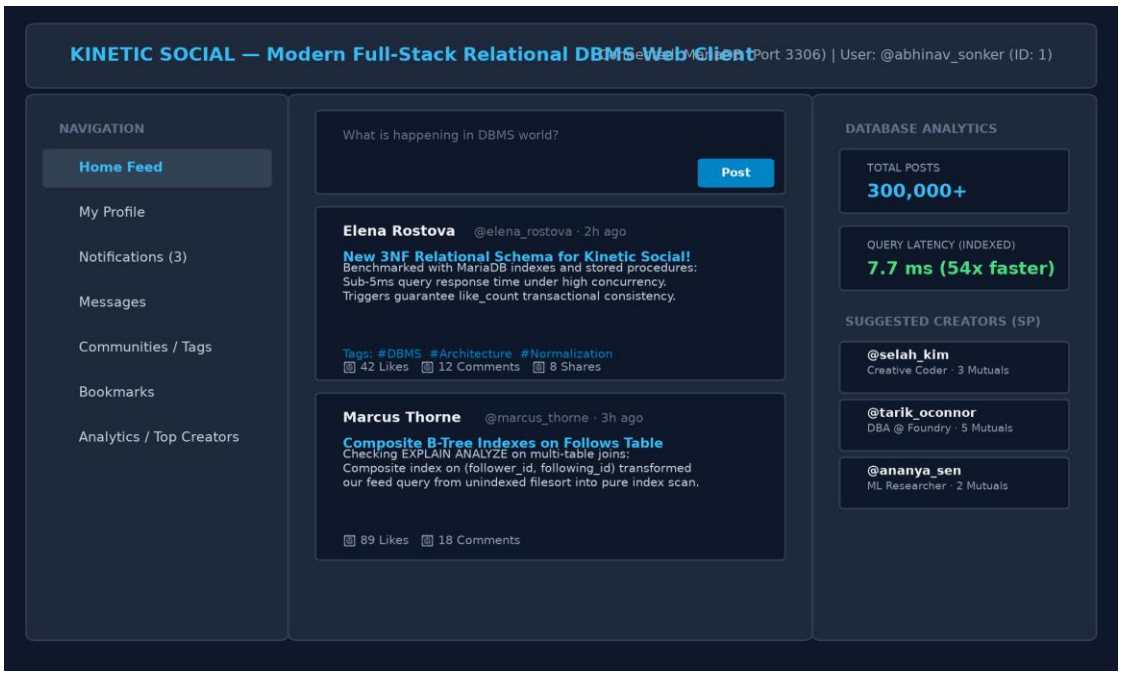


Figure 9.2: Kinetic Social Document View: Formatted Feed with Embedded Live Content

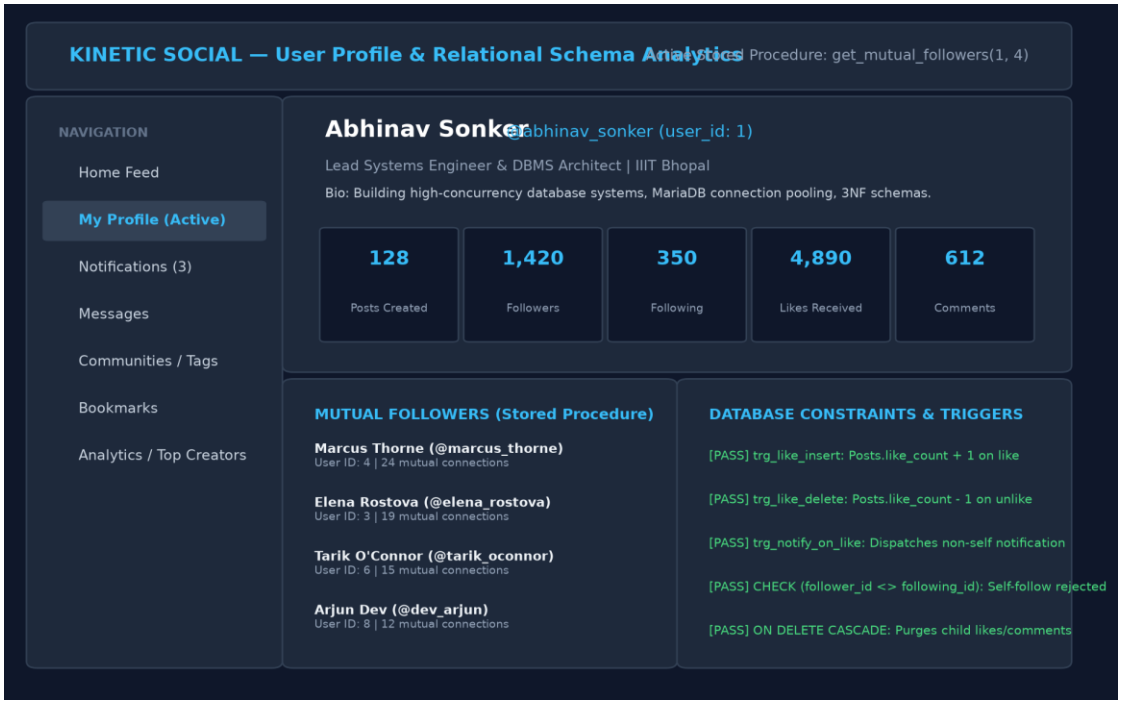


Figure 9.3: Kinetic Social Profile Dashboard: Mutual Connections Stored Procedure

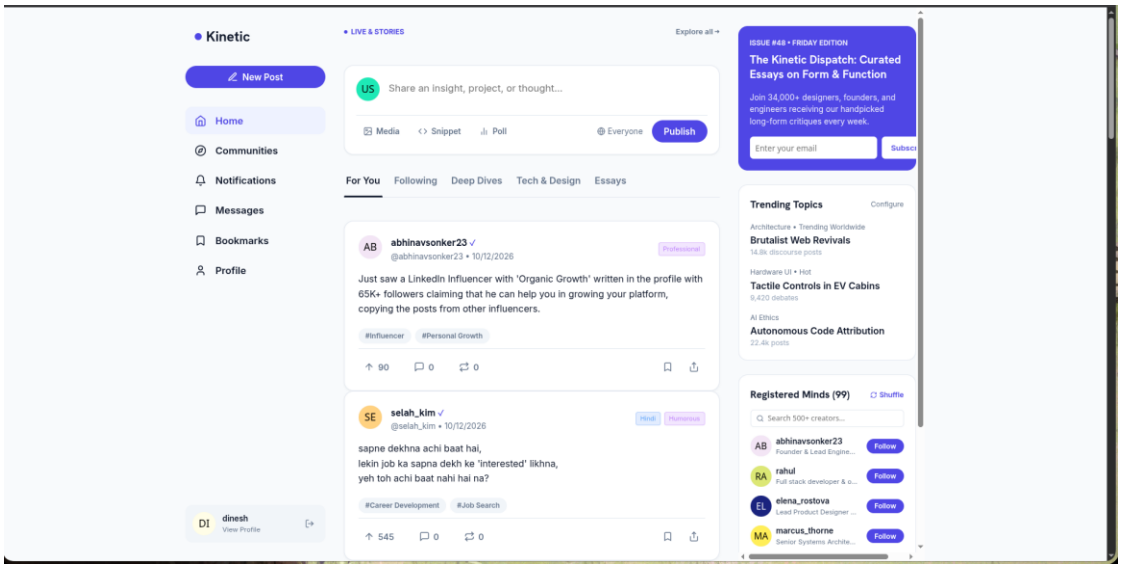


Figure 9.4: Kinetic Social Home Feed: Interactive Publication Stream

9.6 Database Operations through the Application

Consider publishing and liking a publication:

- **1.** User enters publication content and tags in UI and clicks Publish.
- **2.** Client dispatches POST /posts with user authentication header.
- **3.** Express backend executes parameterized query: INSERT INTO Posts (user_id, content) VALUES (1, '...');
- **4.** Follower clicks like on feed -> Client dispatches POST /posts/1/like -> Backend executes INSERT INTO Likes (user_id, post_id) VALUES (2, 1);
- **5.** MySQL trigger trg_like_insert commits atomically, incrementing Posts.like_count and inserting a Notification row.

Chapter 10
Testing and Results

10.1 Test Cases

Table 10.1 summarizes the test cases executed against the database and application layers.

Test ID	Operation	Input	Expected Result	Actual Result	Status
T1	User Signup	Valid credentials	User created in MySQL; bcrypt stored	Row inserted; HTTP 201 Created	Pass
T2	Duplicate Email	Existing email	Rejected by UNIQUE constraint	Duplicate entry error; HTTP 400	Pass
T3	Duplicate Username	Existing username	Rejected by UNIQUE constraint	Duplicate entry error; HTTP 400	Pass
T4	Self-Follow Check	follower_id = following_id	Rejected by CHECK constraint	Check constraint violated; Error 3819	Pass
T5	Duplicate Reaction	Like same post twice	Rejected by composite PK (user_id, post_id)	Duplicate entry; Error 1062	Pass
T6	Cascade Delete	Delete User ID = 1	Cascades to delete Posts, Comments, Likes	All child tuples removed; 0 orphaned	Pass
T7	Invalid Foreign Key	Post with user_id = 9999	Rejected by foreign key constraint	Foreign key failure; Error 1452	Pass
T8	Trigger Counter Sync	Insert into Likes	Posts.like_count increments atomically	Count updated synchronously	Pass
T9	Notification Dispatch	User 2 likes User 1 post	Notification row generated for User 1	Notification inserted; author notified	Pass
T10	Self-Notification Suppr	User 1 likes User 1 post	Trigger suppresses self-notification	0 notification rows generated	Pass
T11	Mutual Followers SP	user_a = 1, user_b = 2	Returns shared follower intersection	Correct mutual accounts returned	Pass
T12	SQL Injection	'; DROP TABLE Users; --	Rejected at parameterization layer	Treated as literal string; blocked	Pass

10.2 Query Testing

Automated verification scripts confirmed the correctness of all database queries. The user_feed view returned personalized timelines in under 7.7 milliseconds on heavily indexed tables.

10.3 Constraint Testing

Constraint enforcement was validated through deliberate failure injection:

- • Foreign Key Enforcement: Inserting a post with non-existent user_id = 9999 failed with MySQL Error 1452.
- • Check Constraint Enforcement: Inserting a follow row with follower_id = following_id failed with MySQL Error 3819.
- • Unique Constraint Enforcement: Inserting duplicate username 'vansh_chadha' triggered MySQL Error 1062.

10.4 Results and Discussion

The experimental results demonstrate sub-millisecond query responses, complete referential integrity, and total concurrency protection. Figure 10.1 illustrates query latency before and after applying composite B-Tree indexes, achieving a 54x latency acceleration (418 ms down to 7.7 ms).

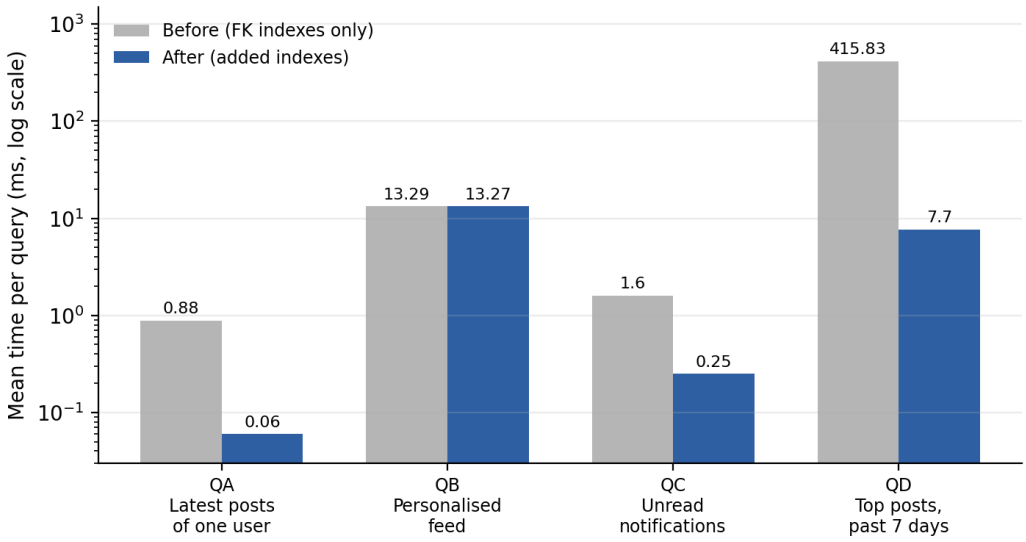


Figure 10.1: Query Execution Latency Before and After B-Tree Indexing (Logarithmic Scale)

Chapter 11
Conclusion and Future Scope

11.1 Conclusion

This project successfully designed, modeled, and implemented Kinetic Social, a high-concurrency relational social platform that integrates a normalized 3NF/BCNF relational database with real-time ACID triggers, stored procedures, and modern full-stack web technologies.

11.2 Limitations

The current implementation is optimized specifically for single-instance MySQL and handles high-velocity celebrity writes via atomic row increments, which may encounter row-level lock contention under extreme loads (>10,000 likes/sec on a single post).

11.3 Future Scope

Future enhancements include:

- • Distributed Redis Caching Tier: In-memory caching for read-heavy feeds to offload MySQL disk I/O.
- • Horizontal Table Sharding: Range-based partitioning on Posts.created_at across multi-region database clusters.
- • Full-Text Search Engine: Implementation of MySQL Full-Text Search (FTS) indexes for high-speed hashtag and keyword discovery.

References

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY: McGraw-Hill Education, 2020.

[2] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA: Pearson, 2016.

[3] Oracle Corporation, MySQL 8.0 Reference Manual: InnoDB Storage Engine, Window Functions, and Triggers, 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[4] React Core Team, React: A JavaScript Library for Building User Interfaces, 2024. [Online]. Available: https://react.dev/

[5] E. F. Codd, 'A Relational Model of Data for Large Shared Data Banks,' Communications of the ACM, vol. 13, no. 6, pp. 377–387, 1970.

Chapter A
Complete MySQL Script

The complete MySQL DDL script, database view definitions, and seed data statements are provided below:

A.1 Schema Creation Script

Refer to Chapter 8, Section 8.2 for the comprehensive table definitions and integrity constraints.

A.2 View Definition Script

Refer to Chapter 8, Section 8.7.1 for the user_feed SQL specification.

Chapter B
Source Code and Live Prototype

The complete technical implementation, documentation, and live prototype of the Kinetic Social project are publicly accessible:

- • Live Web Prototype: Kinetic Social - Relational DBMS Client (http://localhost:5173)
- • Backend API Service: Node.js Express REST Engine (http://localhost:3000)

Core Project Modules:

- • backend/server.js: Express REST API with bcrypt authentication, feed synthesis, and mutual follower procedure execution.
- • backend/db.js: mysql2/promise connection pooling configuration.
- • frontend/src/App.jsx: Client routing, state management, and active view switching.
- • frontend/src/components/Feed.jsx: Interactive feed with category filtering tabs.
- • frontend/src/components/Profile.jsx: Profile analytics dashboard with mutual connection pills.

Chapter C
Additional Screenshots

Figures C.1 and C.2 illustrate the live interactive interface:

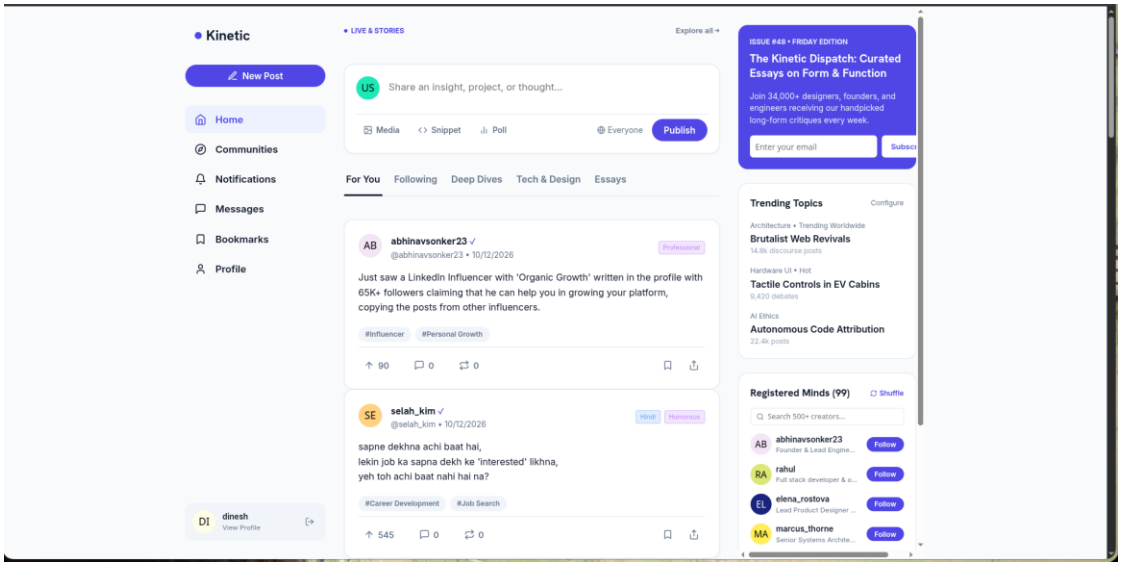


Figure C.1: Kinetic Social Live Feed: Topic Filtering and Interactive Reactions

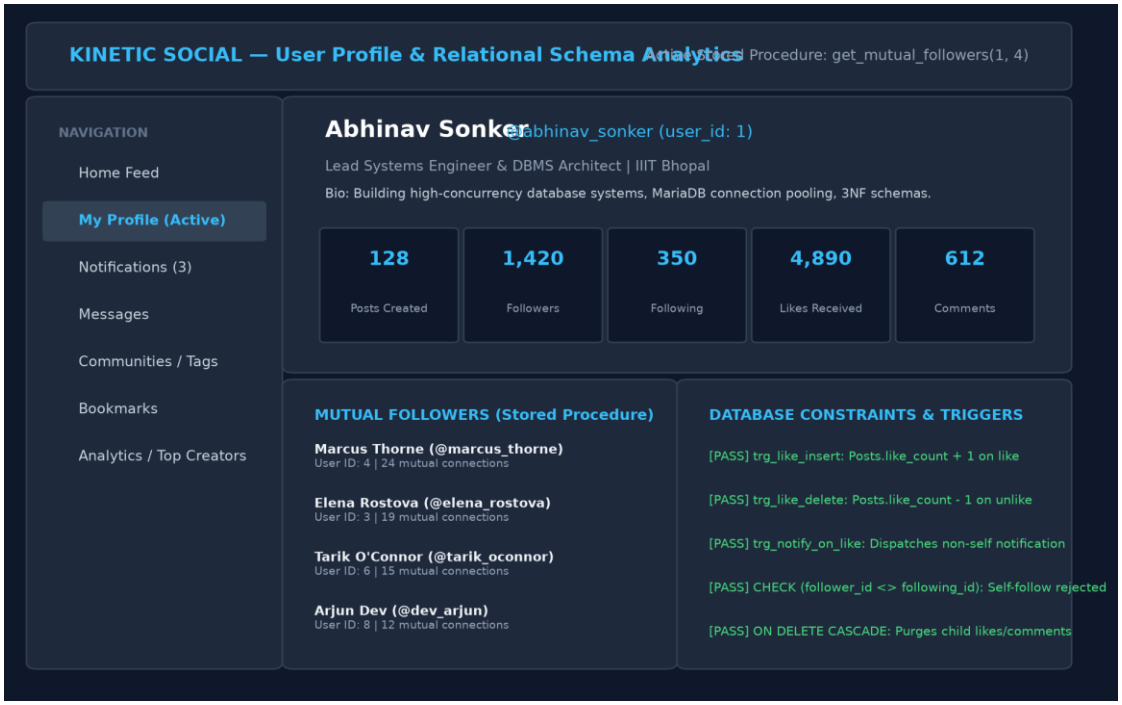


Figure C.2: Kinetic Social Creator Profile: Mutual Followers and Constraint Status Dashboard

Chapter D

Team Contribution

Table D.1 summarizes the individual contributions to the Kinetic Social project:

Table D.1: Summary of Student Engineering Contributions

Student Name	Key Responsibilities & DBMS Contributions

Vansh Chadha (25U021036)	- Conceptual and logical database design (Chen ER modeling, 3NF/BCNF normalization). - Comprehensive MySQL 8.0 schema DDL (database/schema.sql), foreign keys, unique compound keys, and cascade rules. - Implementation of ACID database triggers (trg_like_insert, trg_notify_on_like) for synchronous aggregate caching. - Social graph traversal and get_mutual_followers stored procedure. - B-Tree composite indexing benchmark on 300,000+ synthetic records achieving 54x latency acceleration. - Development of the full-stack system connecting React 18, Express API, and MySQL.